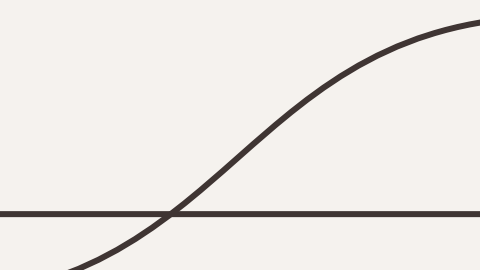


Arrow Dx. Update Presentation

Summer 2024 - Yasha Hasija, Arvind Swamynathan, Logan Krause



Previous Interns' Work

- After a comprehensive literature review, last summer's interns decided upon using a CNN (Convolutional Neural Network) for the efficient classification of Raman spectroscopy data
 - Datasets used for training and testing consisted of Raman spectra for nine mixtures of two analytes: thiophenol and alkanethiol
 - Corresponding intensities of 1015 wave numbers were recorded for each mixture, and such measurements were repeated 51 times
 - The developed CNN worked with 99% validation accuracy to classify individual spectra measurements into categories corresponding to the number of carbon atoms in the alkanethiol chain

Current Work

Models We've Tried:

- K-Means Clustering
- Decision Trees
- K-Nearest Neighbors
- Random Forest
- VAEs

Optimization

- Max pooling
- Adjusting batch size
- Adjusting learning rate
- Adjusting number of epochs

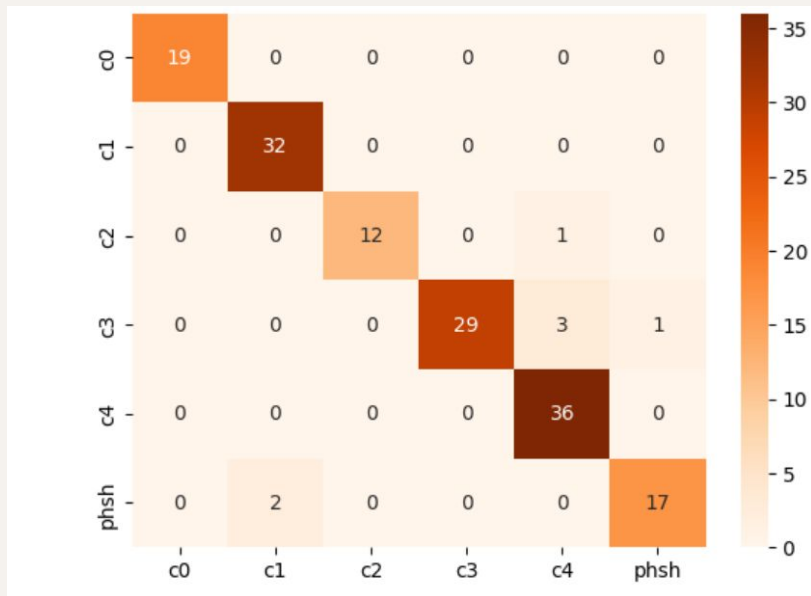
Data Used:

- Spectra of 9 mixtures and 2 analytes (as described earlier)
- SERS data for 63 metabolites
- Bootstrapping (random generation) of data points using current data

Obstacles

- Algorithm is not universal, trained solely on the mixture data
- When tested again, the model did not provide 99% accuracy (max recorded was 96.47%)
- Difficult time finding relevant data (diseased vs. healthy spectra)

Model Visualizations (model 1)



Convolutional Neural Network Heatmap

**note: high accuracy may be due to overfitting, which is most likely due to our limited data set.
additional testing/data would give us a better accuracy!

```
▼ KNeighborsClassifier
KNeighborsClassifier()

neigh.predict(X_test)

neigh.score(X_test, y_test)

0.9802631578947368
```

```
▼ RandomForestClassifier
RandomForestClassifier(max_depth=41, random_state=0)

clf.score(X_test, y_test)

0.9868421052631579
```

k-Nearest Neighbors and Random Forest Classification Results

Bootstrapping

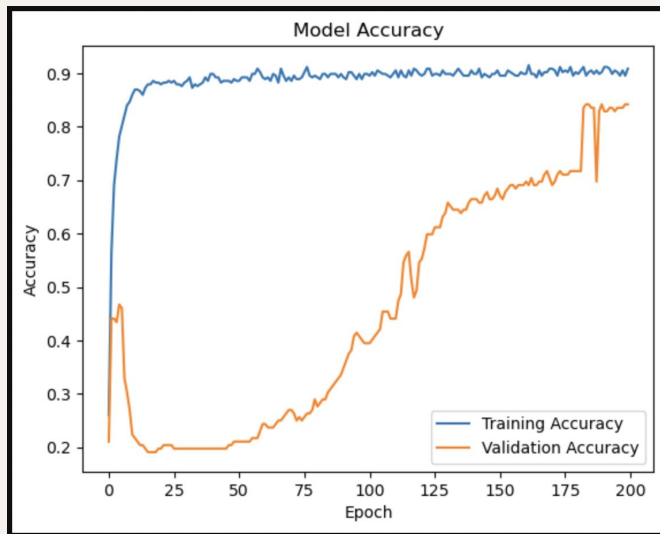
Due to our limited data, we tried to bootstrap, generating random data from the SERS readings

This model was also normalized by each molecule's columns rather than the collective columns (allows for representativeness)

*note accuracy drops with this type of normalization, but it may prevent overfitting

Model Accuracy

Randomly generated data's accuracy



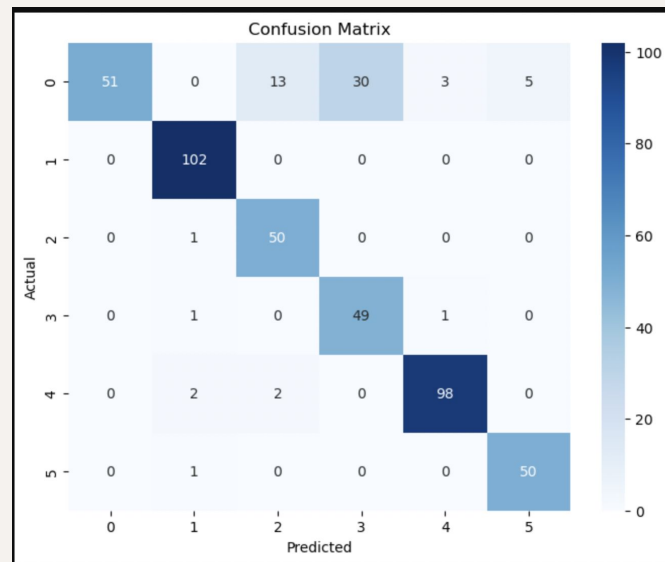
```
model.evaluate(X_test, y_test)
```

```
5/5 [=====] - 0s 2ms/step - loss: 1.4327 - accuracy: 0.8421
```

```
from sklearn.metrics import accuracy_score  
y_pred_classes = np.argmax(y_pred, axis=1)  
y_rand_classes = np.argmax(y_rand, axis=1)
```

```
print(accuracy_score(y_rand_classes, y_pred_classes))
```

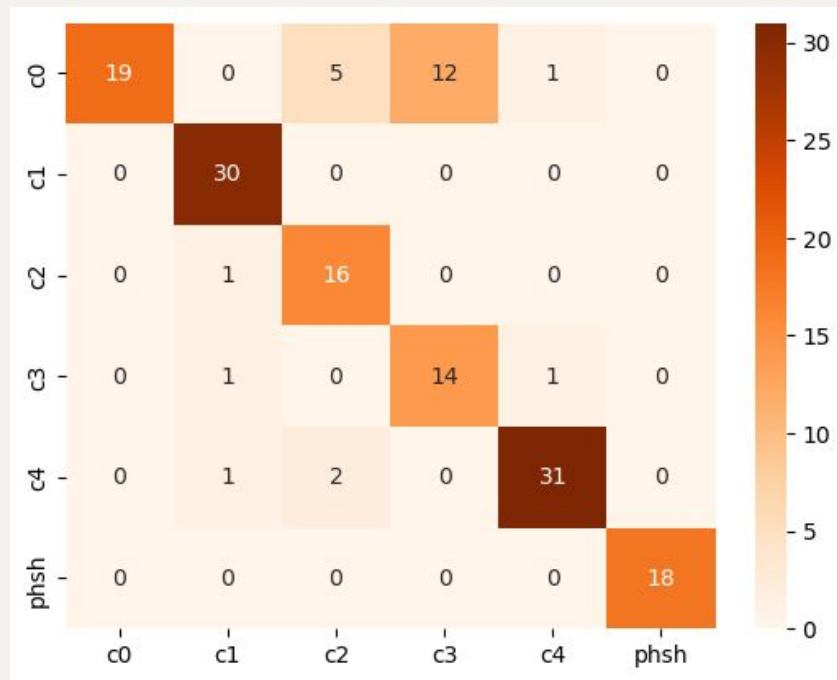
```
0.8714596949891068
```



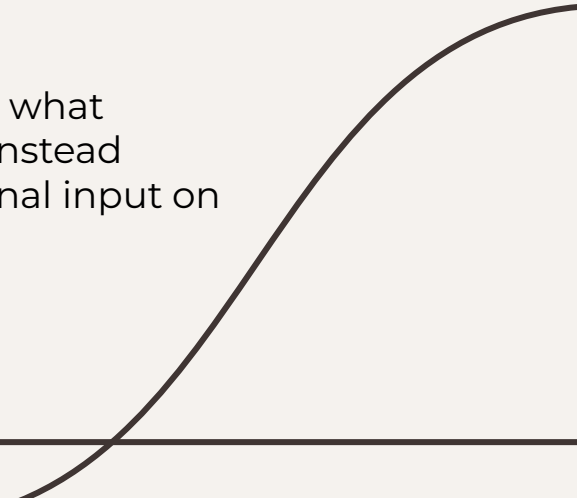
Bootstrapping cont.

Heat map of the 6 layer molecule normalized model (normalization was applied for each molecule rather than the whole data set)

Has lower accuracy (84%) than model from slide 4, but this method may avoid overfitting and yields high accuracy on randomly generated data

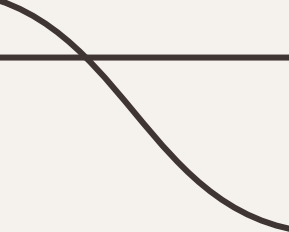


What we want

- Datasets! Datasets! Datasets! - Right now the big thing we are lacking is access to organically produced blood datasets that have raman spec. We have access to other datasets such as pure Raman specs however, it will increase overall model effectiveness to have this data.
 - Randomly generated data possibility
 - Specialization Focus - We need to start narrowing down what particular thing we are trying to optimize, disease wise instead general molecule detection, and we could use professional input on best choices!
- 

How you can help!

- **Is it possible for us to find/be provided with SERS data for blood samples (ideally that of healthy and diseased patients suffering from NAFLD) to help optimize a model for this specific problem?**
 - Or, should we continue training our model for the identification of molecules and metabolites?
 - Is there anything we may be overlooking in terms of the data processing/cleaning process?
 - Would you recommend us to tackle this problem in another way?
- 



Thank you!

We hope to follow up with our developments with you soon!

